ASSIGNMENT 6 — QUESTION 5

Q5 - Frontend Rush: Avoiding Callback Hell

JavaScript (ES6+) solution with comments

```
// Q5A: Nested callbacks (callback hell demonstration).
function design(callback) {
    setTimeout(() => { console.log("Design complete"); callback(); }, 1000);
}
function build(callback) {
    setTimeout(() => { console.log("Build complete"); callback(); }, 1000);
}
function test(callback) {
    setTimeout(() => { console.log("Tests passed"); callback(); }, 1000);
}
function deploy(callback) {
    setTimeout(() => { console.log("Deployment complete"); callback(); }, 1000);
}
function celebrate(callback) {
    setTimeout(() => { console.log("Celebrate!"); if (callback) callback(); }, 1000);
}

console.log("Callback version:");
design(() => {
    build(() => {
        test(() => {
            deploy(() => {
                celebrate(() => console.log("Pipeline finished."));
            });
        });
    });
});

// Q5B: Async/await version.
// Each stage is still asynchronous, but the sequence is easier to read.
const waitOneSecond = () =>
    new Promise(resolve => setTimeout(resolve, 1000));

async function runStage(name, message) {
    await waitOneSecond();
    console.log(message);
}

async function runPipeline() {
    console.log("Async/await version:");
    await runStage("design", "Design complete");
    await runStage("build", "Build complete");
    await runStage("test", "Tests passed");
    await runStage("deploy", "Deployment complete");
    await runStage("celebrate", "Celebrate!");
    console.log("Pipeline finished.");
}

runPipeline().catch(error => console.error("Pipeline failed:", error));

/*
Why async/await is clearer:
- The steps read from top to bottom like ordinary sequential code.
- It avoids deeply nested callback indentation.
- try/catch can handle errors in one familiar place.
*/
```